TASK 03 – Prompting for Task Automation

Prompt Engineering Internship

This task uses prompting to automate the extraction of structured information from everyday, unstructured text. A single reusable prompt is designed to convert short expense notes into a consistent JSON format, and is tested against three different inputs to check reliability.

Automation Task: Converting Expense Notes into Structured JSON

Reusable Automation Prompt

You are a finance data extraction assistant. Read the given text and extract the available information into a valid JSON object. Use the following fields: spender, item, amount, category, date. If any information is missing, use "Not specified" instead of guessing. Keep the original meaning of the input and return only valid JSON without additional explanation.

Input 1

Input:

Priya spent Rs. 450 on groceries yesterday.

Output

```
{
 "spender": "Priya",
 "item": "Groceries",
 "amount": "Rs. 450",
 "category": "Groceries",
 "date": "Yesterday"
}
```

Analysis

The prompt correctly identifies all five fields from a short, casual sentence and converts it into a consistent JSON structure without adding extra commentary.

Input 2

Input:

Karthik paid for a movie ticket, around 250 rupees, on 15th August. It was for entertainment.

Output

```
{
 "spender": "Karthik",
 "item": "Movie ticket",
 "amount": "250 rupees",
 "category": "Entertainment",
 "date": "15th August"
}
```

Analysis

The same prompt handles a differently phrased input with the fields in a different order, showing that the prompt is reusable across varied sentence structures rather than tied to one fixed format.

Input 3

Input:

Anjali bought a notebook for college. She hasn't mentioned the price yet.

Output

```
{
 "spender": "Anjali",
 "item": "Notebook",
 "amount": "Not specified",
 "category": "Not specified",
 "date": "Not specified"
}
```

Analysis

The prompt correctly handles missing information. Since the price, category, and date are not mentioned, it returns "Not specified" instead of inventing values, which keeps the automated output trustworthy.

Summary of Test Results

Input	Fields Extracted Correctly	Missing Fields Handled	Valid JSON Returned
Input 1	5 / 5	N/A	Yes
Input 2	5 / 5	N/A	Yes
Input 3	2 / 5	3 fields marked 'Not specified'	Yes

Prompt Iteration and Debugging

In an early version of the prompt, the model sometimes added a short explanation before or after the JSON output, such as "Here is the extracted data:". To fix this, the instruction "return only valid JSON without additional explanation" was added.

A second issue appeared when a field was missing from the input — the model occasionally guessed a plausible value instead of leaving it blank. Adding the explicit rule "If any information is missing, use 'Not specified' instead of guessing" resolved this and made the output safe to use without manual review.

Key Learnings from Task 03

- Reusable Prompts – A well-designed prompt can process many different inputs without being rewritten each time.
- Consistent Output – Defining a fixed field structure keeps results predictable across varied input formats.
- Handling Missing Data – Explicit fallback instructions stop the model from guessing unavailable information.
- Output Validation – Requiring strictly valid JSON makes the output directly usable in downstream applications.
- Prompt Debugging – Testing with multiple inputs surfaces edge cases like missing fields or unwanted explanations.
- Task Automation – Prompting can turn repetitive manual data entry into a single reusable extraction step.

Conclusion

This task demonstrates how prompt engineering can automate a small but genuinely useful real-world task. A single, carefully worded prompt reliably converted three differently phrased expense notes into a consistent structured format, including correct handling of missing data. Iterating on the prompt after observing its failure cases — extra text and guessed values — was key to making the automation dependable.